Title: Deployment Environment and Cloud Platform Recommendation: Municipal Resident-Request Assistant

Name: Justin Williams

Student ID: 100932604

Course: INFT 41000 – Deploying Artificial Intelligence Systems

GitHub Commit Link:

AI-Use Statement: Gemini was used as a brainstorming tool to help structure the report analysis, draft scorecard summaries, and organize cost-comparison templates. Grammarly was used to check spelling. All model and service names, pricing information, and architectural decisions were manually verified against the relevant vendor documentation.

Section 1: Use case and readiness snapshot

Use case Description

Target Users: Residents, city staff, public workers, service agents

Action: Automatically group and route incoming resident service requests (i.e., potholes, transit issues) and provide consistent updates on a 24/7 basis.

Data Sources: Resident submissions (unstructured text, municipal tickets, geographic GIS coordinates, service catalogues)

Model Type: Natural language understanding / intent classifier, large language model, RAG pipeline, computer vision model for image classification.

Lifecycle Stage: Production serving, model monitoring for hallucinations and continuous intent accuracy tuning.

1.2 Production AI

Municipal requirements cannot be met with a simple prototype. The resident assistant is on call 24/7 to handle unpredictable surges in volume from severe winter storms or emergencies. The consequences include delayed emergency response, wasted city maintenance resources, public dissatisfaction and exposure of municipal and citizen data because of an unreliable system or incorrect routing.

1.3 Score the six readiness dimensions

Dimension	Score (0-5)	Evidence (One specific Sentence)	Effect on the deployment decision
Data	3	There are plenty of municipal logs, but the resident comments often contain personal health information or identifiable information that needs to be automatically masked.	Monitor after launch
Model	4	Open-source LLM base models and classification heads that can achieve on high levels of accuracy on municipal datasets	Fix before launch

Infrastructure	2	City hall infrastructure runs on-premises servers that can lack of hardware configuration (i.e., GPUs) needed for modern NLP/LLM inference.	Fix before launch
Organizational	2	City's IT team is small and focuses on the city's infrastructure, but it has no dedicated MLOps engineers on staff	Fix before launch
Risk	3	The municipal has a strict compliance standard that must locate citizen data within domestic boundaries.	Fix before launch
Cost	2	Upgrades the local servers are strictly limited by Capital expenditures due to the tight municipal budget.	Fix before launch

1.4 Identify the two lowest-scoring dimensions

1. Organizational (Score 2): No dedicated MLOps staff, so the decision is to use managed PaaS/SaaS services instead of custom, self-managed Kubernetes infrastructure.

2. No GPU hardware and a constrained capital budget mean the workload cannot purchase on-premises hardware and must use pay-as-you-go cloud compute.

Section 2: Deployment environment evaluation

2.1 Complete the Environment Fit Scorecard

Question (In order)	Facts about my use case	Score (1 = on-premises, 5 = cloud)
1. Data Sensitivity	The data must retain within a specific geographic region, so that it can be relatively low risk because it can consist of non-classified public service tickets that with personally information removed.	3

2. Cost Pattern	Severe storm, for instance, can cause ticket submission spikes that are sudden and unpredictable, making adaptable cloud auto scaling a great fit.	4
3. Operational Capacity	The municipal IT team is small and can't deal with complex hardware pipelines so that are pushing hard for fully managed cloud services.	5
4. Latency	Cloud round trip times are readily compatible with asynchronous and near real-time chat expectation (2-3 seconds)	4

Average: 4.0

Spread: 2

Rule says: no limit was triggered by 1. The average of 4.0 indicates that it should be a cloud deployment.

2.2 Three-Environment Comparison

On-Premises: Full control over privacy of data but high upfront Capex for GPUs, limited scalability during storm surges and a high maintenance burden for a small IT team.

Cloud: Adaptive scalability, no local hardware overhead, low operational burden; strict regional locking needed to satisfy municipal information residency requirements

Hybrid: The network routing is complex, yet there is no need for overhead since municipal ticketing has no strict latency limits under 50ms.

2.3 Deployment Recommendation & Justification

Recommendation: Cloud

Reason: Cloud deployment is the best choice given municipal constraints. It removes the need for hardware and lets the provider manage the MLOps infrastructure. By using public sector cloud zones in the area, data residency requirements are met. This matches the environment fit scorecard results.

2.4 Three-Year Cost Reasoning

Option A (Cloud - Recommended): Use a serverless or PaaS setup, costing $65,000 over three years. This covers API calls, serverless computing, regional S3 storage, and low administrative overhead.

Option B (On-Premises Alternative): Buy on-premises servers with GPU modules, power and cooling, and hire specialized contractors for maintenance. This would cost $160,000 over three years.

The biggest uncertainty is how much the public will use large language model inference and tokens during an unexpected municipal emergency.

2.5 Tipping Point

If the regional government passes privacy laws that require citizen interaction data to be stored physically at city hall, the deployment will need to switch to an on-premises model.

Section 3: Cloud Platform and service selection

3.1 Choose one platform: AWS (Amazon Web Services)

Why: AWS provides dedicated local regions that meet municipal data residency laws, strong public-sector compliance, and has lower-cost serverless AI options like AWS Bedrock.

Rejections:

Azure: Has a higher baseline cost structure that is not suitable for small-scale deployments under municipal tight budgets

Google Cloud: It has fewer localized public sector framework contracts that are within the municipality's regional jurisdiction

3.2 Build a service map

Building Block	Named Service	Model	Container/Serverless	Why
Training Compute	AWS Sagemaker	PaaS	Container	Support recurring batch fine-tuning of ticket-categorization models.
Serving/Hosting	AWS Bedrock	PaaS	Serverless	Provides serverless container for the RAG conversational assistant
Data Storage	AWS S3	PaaS	N/A	Store resident image uploads and archived logs, with lifecycle rule to manage data retention
Managed AL / ML	AWS Bedrock	PaaS	Serverless	Provides managed access to LLM for the intent of detention and drafting resident response.
Additional (security Service) AWS	AWS Comprehend	PaaS	Serverless	Protect and process resident data. KMS encrypts data, while

Key Management Service and amazon Comprehend				comprehend automatically identifies and masks personal information in resident text.

3.3 Containers or serverless

The components that we are serving for this system are serverless, because we are using AWS ECS fargate/Lamba due to the unpredictable nature of the request pattern we could face, eliminating pay-for-idle costs.

3.4 Build vs. Buy

The decision is to buy/manage the service stack using AWS Bedrock and PaaS because building a self-managed LLM stack requires dedicated infrastructure that a small IT team lacks. The services that we are managing can make the process happen faster in time to value and keep ongoing operational costs low.

3.5 Cloud Adoption Framework

By using the CAF (Cloud Adoption Framework), the framework has PRAGO, also known as plan, ready, adopt, and govern & operate.

- Plan: Create a map of all service SLAs, create a privacy impact assessment (PIA and define a success metric
- Ready: Configure all AWS organizations with guardrails, IAM role-based access that can be controlled, and have strict budget alert thresholds to ensure that, in the long term, you are keeping your operational cost down low.
- Adopt: Integrate an intake web form that has AWS API Gateway and can run acceptable testing with city staff.
- Govern & operate: Create automated compliance audits, monitor any consumption, and review any response accuracy metrics on a weekly basis.

3.6 Lock-In Assessment

The point of highest lock-in is using the AWS Bedrock proprietary model because it mitigates the need for a wrapper model in an open-source layer (i.e., LangChain or a custom wrapper) to allow switching model providers easily.

Section 4: Risks and decision

4.1 Name the top three risks

Top Three Risk	Why?	Rating Likelihood	Impact	Mitigation
Hallucination / Misinformation Risk	Assistant can provide a resident with inaccurate data or guidance about certain city bylaws or services.	Medium	High	Using a strict RAG for official city documents and will need a human to review any ticket/query that is unclear or sensitive.
Data Residency Compliance Branch	Personal Information could be uploaded without unauthored cloud regions.	Low	High	Use AWS to detect and remove any personal information on the client side and ensure storing the data in the cloud can be approved in regional availability zones.
Unexpected Budget Overrun (Cost)	An Automated bot can generate unnecessary requests. In resulting, a high cloud-service and token costs.	Medium	Medium	Can developed a rate limiting through AWS WAF and found firm billing cap with automated throttling.

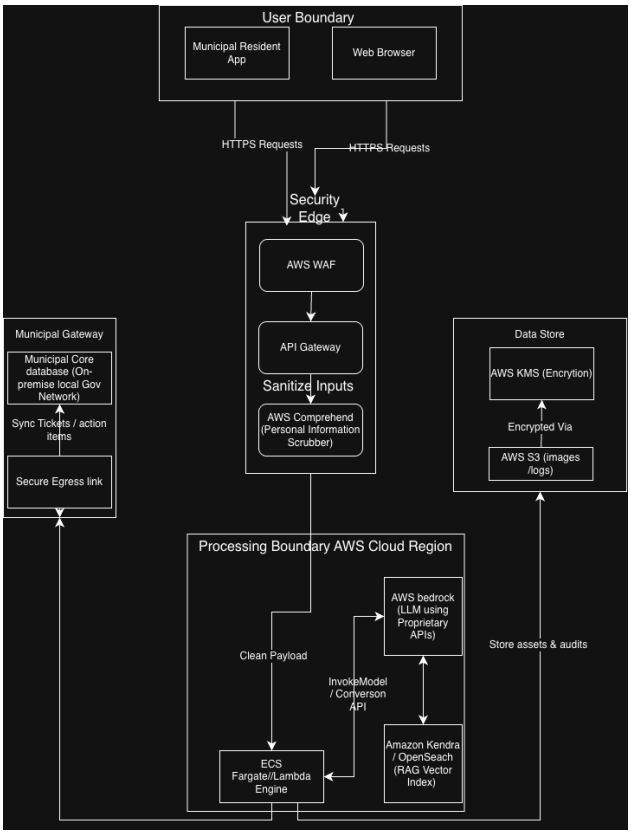
4.2 Go, Delay, No-Go decision

Decision: Will proceed with the launch, subject to the following rules will apply:

1. Municipal legal counsel must complete and sign off on the privacy impact assessment or known as PIA.
2. The automated Personal information stripping process must be validated to any dataset and demonstrate at least an 99% accuracy.

Section 5: Communication and professional practice

5.1 Deployment Diagram structure



5.2 Required References

Amazon Web Services. (2026). *AWS Bedrock and Cloud Compliance Frameworks*. AWS Documentation. https://aws.amazon.com/bedrock/security-compliance/

Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (2016). Site Reliability Engineering: How Google Runs Production Systems. O'Reilly Media.

Huyen, C. (2022). *Designing Machine Learning Systems: An Iterative Process for Production-Ready Applications*. O'Reilly Media.

Mell, P., & Grance, T. (2011). *The NIST Definition of Cloud Computing* (NIST Special Publication 800-145). National Institute of Standards and Technology. https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-145.pdf

Appendix

Appendix A: Completed Readiness Scorecard

Dimension	Score (0-5)	Evidence (One specific Sentence)	Effect on the deployment decision
Data	3	There are plenty of municipal logs, but the resident comments often contain personal health information or identifiable information that needs to be automatically masked.	Monitor after launch
Model	4	Open-source LLM base models and classification heads that can achieve on high levels of accuracy on municipal datasets	Fix before launch
Infrastructure	2	City hall infrastructure runs on-premises servers that can lack of hardware configuration (i.e., GPUs) needed for modern NLP/LLM inference.	Fix before launch
Organizational	2	City's IT team is small and focuses on the city's infrastructure, but it has no dedicated MLOps engineers on staff	Fix before launch
Risk	3	The municipal has a strict compliance standard that must locate citizen data within domestic boundaries.	Fix before launch
Cost	2	Upgrades the local servers are strictly limited by Capital expenditures due to the tight municipal budget.	Fix before launch

Appendix B:

Table B1: Environment Fit Scorecard Summary

Question (In order)	Facts about my use case	Score (1 = on-premises, 5 = cloud)
1. Data Sensitivity	The data must retain within a specific geographic region, so that it can be relatively low risk because it can consist of non-classified public service tickets that with personally information removed.	3
2. Cost Pattern	Severe storm, for instance, can cause ticket submission spikes that are sudden and unpredictable, making adaptable cloud auto scaling a great fit.	4
3. Operational Capacity	The municipal IT team is small and can't deal with complex hardware pipelines so that are pushing hard for fully managed cloud services.	5
4. Latency	Cloud round trip times are readily compatible with asynchronous and near real-time chat expectation (2-3 seconds)	4

Average: 4.0

Spread: 2

Rule says: no limit was triggered by 1. The average of 4.0 indicates that it should be a cloud deployment.

Table B2: Environment Comparison for my use case

Dimension	On-Premises	Cloud	Hybrid
Data Control	Maximum physical control over data on-site but it creates local security burden for legacy city system.	Supervised via cloud security controls and strict regional data residency locking like AWS Canada/Regional availability zones.	Splitting the data control can be sensitive citizen logs that can stay local while streams to cloud increasing the compliance monitoring involvement.

Cost Model	High upfront capital expenditure (i.e. CapEx) for server hardware and GPU modules that the city budget cannot support.	Flexible operational expenditure (OpEx) with pay-as-you-go serverless compute that can be ideal for tight municipal budgets	Dual cost structure that requires local CapEx for host server alongside with the ongoing cloud OpEx fees.
Scalability	Fixed hardware ceiling that can be vulnerable to server crashes or slow response times during municipal storm surges	Flexible auto-scaling handles massive that can be unpredictable resident ticket spikes during emergencies without crashing	The involvement scaling can have local ingestion bottlenecks limit that can total cloud processing speed during traffic spikes.
Operational Burden	High operational overhead, it requires dedicated hardware maintenance that a small municipal IT team cannot support	Minimal operational burden that the infrastructure patching, hardware maintenance, and uptime are manage by the cloud provider	High operational burden that requires maintaining two zones' environments with zero assign MLOps staffs.
Best fit for my system	No fails cost and scalability constraints	Yes, best fit for this municipal budget, small IT team and storm spikes	No, it adds to unnecessary architectural involvement without benefit.

Appendix C

Table C1: Service Map (AWS Platform)

Building Block	Named Service	Model	Container/Serverless	Why
Training Compute	AWS Sagemaker	PaaS	Container	Support recurring batch fine-tuning of ticket-categorization models.
Serving/Hosting	AWS Bedrock	PaaS	Serverless	Provides serverless container for the RAG conversational assistant
Data Storage	AWS S3	PaaS	N/A	Store resident image uploads and archived logs, with lifecycle rule to manage data retention
Managed AL / ML	AWS Bedrock	PaaS	Serverless	Provides managed access to LLM for the

				intent of detention and drafting resident response.
Additional (security Service) AWS Key Management Service and amazon Comprehend	AWS Comprehend	PaaS	Serverless	Protect and process resident data. KMS encrypts data, while comprehend automatically identifies and masks personal information in resident text.

Table C2: Three-Year Comparison (Order of Magnitude)

Cost Line	Assumption and Source	Option A: Cloud (AWS)	Option B: On-Premises	Uncertainly
Compute	AWS Pricing Calculator (Fargate + Bedrock API token) vs Server HW	$25,000	$75,000	Medium
Storage & Transfer	AWS S3 standard regional pricing	$8,000	$15,000	Low
Managed Service	AWS Comprehend & KMS Usage fees	$12,000	$20,000	Medium
Staff Time To Operate	Internal allocation (0.2 FTE Cloud vs 1.0 FTE On-prem maintenance)	$20,000	$50,000	Low
Three-Year Total	Order-of-magnitude	$65,000	$160,000	Medium